

Greedy technique
↓
dynamic program

L-49

selection

merge sort

25/11

6-8

Lec_50: Insertion Sort

30/11/2020
D.S.

Ex:

70, 80, 83, 42, 81, 100, 76, 4, 90

sort in increasing order using Insertion sort?

solⁿ
arr = { 70, 80, 83, 42, 81, 100, 76, 4, 90 }

{ 70 } ✓

{ 70, 80 } ✓

{ 70, 80, 83 }

{ ~~70~~, ~~80~~, ~~83~~, 42 }

42 42 42 83
70 80

{ 42, 70, 80, 83 }

{ 42, 70, 80, ~~83~~, ~~81~~, 83 }

{ 42, 70, 80, 81, 83, 100 }

{ 42, 70, ~~80~~, ~~81~~, ~~83~~, 100, 76 }

76 76 76 76 100
80 81 83

{ ~~42~~, ~~70~~, ~~76~~, ~~80~~, ~~81~~, ~~83~~, ~~100~~, 4 }

4 4 4 4 4 4 100
42 70 76 80 81 83

{ 4, 42, 70, 76, 80, 81, 83, 100 } 90
90
100

(4, 42, 70, 76, 80, 81, 83, 90, 100)
 Now sorted

(⁰70, 80, 83, 42, 81, 100, 76, 4, 90)ⁿ⁻¹

Let size of array is n

```

for (i = 0; i < n-1; i++)
{
  for (j = i; j < n; j++)
  {
    if (a[j-1] > a[j])
    {
      swap(a[j-1], a[j])
    }
  }
}

```

$\frac{n(n+1)}{2} = O(n^2)$

Algo:

```
for( i=0; i<=n-1; i++)  
  for( j=i; j<=n-1; j++)  
    if( a[j-1] > a[j])  
      swap( a[j-1], a[j])  
}
```

$O(n^2)$

10 20 30 40 50 -
i 1 2 3 4

```
for( i=0; i<=n-1; i++)  
  j=i  
  while( j<=n-1 and a[j] < a[j-1])  
    swap( a[j-1], a[j])  
    j--
```

Best code

improved

$O(n)$
improvement

10, 20, 30, 40, 50, 60
~~0, 1, 2, 3, 4, 5~~

Enters in
 while loop
 X

for ($i=0$; $i \leq n-1$; $i++$)
 $j=1$

while ($j \geq 1$ and $a(j+1) > a(j)$)

$\{ swap(a[j+1], a[j]) \}$

$i=0$

↑

$j=0$

$i=1$

$j=1$

$i=2$

$j=2$

$O(n)$

$T_c \Rightarrow O(n^2)$, worst case
 avg case

$T_c = O(n)$ in best case

$O(n^2)$
 WC, AC

$O(n)$
 BC

SC
 $O(1)$

→ If array is already sorted insertion sort gives $O(n)$ and also bubble sort (modified) gives $O(n)$ for

Imp points in Sorting:

- To sort n -elements using Insertion sort BC: 0 swaps and $n-1$ comparisons, TC = $O(n)$, (when array is already sorted)
- If array is already sorted Insertion Sort takes $O(n)$ but Quick sort takes $O(n^2)$, but Merge sort does not care always takes $O(n \log n)$
- No of inversions in array is equal to no of swaps in insertion sort ✓
- If array contains n elements and n inversions then to sort that array insertion sort will take $O(n)$ because array is almost sorted
- Insertion Sort is Inplace and ~~Sorted~~ Stable

$$\frac{n(n+1)}{2}$$

Quick, merge, bubble, selection